7) Cancer dataset (30-features). take a perceptron and Work with activation functions Threshold

```
import numpy as np
from sklearn.datasets import
load_breast_cancer
from sklearn.model_selection import train_test_split
# Load Breast Cancer Dataset
data = load_breast_cancer()
X = data.data          # 30-features
Y = data.target        # 0= malignant, 1= benign
# convert labels to -1 and +1 (req for threshold model)
Y = np.where(Y==0,-1,1)
# Train-test split
X_train, X_test, Y_train, Y_test = train_test_split
(X, Y, test_size= 0.2, random_state=42)
# Min-Max Normalization (use Train data only)
X_min = X_train.min(axis=0)
X_max = X_train.max(axis=0)
X_train = (X_train - X_min)/(X_max - X_min)
X_test = (X_test - X_min)/(X_max - X_min)
# Initialize single neuron parameters
np.random.seed(1)
Weights = np.random.rand(30)
bias = 0.0
Learning_rate = 0.01
epochs = 10
# Threshold (step) function
def threshold(x):
    return 1 if x>=0 else -1
# Training phase
print("TRAINING PHASE (THRESHOLD FUNCTION)")
print("=" * 60)
for epoch in range(1, epochs+1):
    correct = 0
    for i in range(len(X_train)):
        net_input = np.dot(X_train[i], weights) +
                    bias
        prediction = threshold(net_input)
        error = Y_train[i]
```

```
    # update weights and bias
    Weights += Learning-rate * error * x-train[i]
    bias += learning-rate * error
    if prediction == y-train[i]:
        correct += 1
epoch - accuracy = (correct / len(X-train)) * 100
print(f" Epoch {epoch}")
print(f" Training Accuracy: {epoch-accuracy:.2f}%")
print(f " Bias: {bias:.4f}")
print(f" Weights (first 5 shown): {weights[:5]}")
print("-" * 60)
# Final Training Accuracy
print("\n Final Training Accuracy")
print("=" * 60)
correct = 0
for i in range(len(X-train)):
    net-input = np.dot(X-train[i], weights) + bias
    prediction = threshold(net-input)
    if prediction == y-train[i]:
        correct += 1
print(f" Final Training Accuracy: {(correct/len(X-train)) *
        100:.2f}%")
# Testing phase
print("\n Testing phase")
print("=" * 60)
correct = 0
for i in range(len(X-test)):
    net-input = np.dot(X-test[i], weights) + bias
    prediction = threshold(net-input)
    if prediction == y-test[i]:
        correct += 1
print(f" Final Test Accuracy : {(correct / len(X-test)) * 100:
        .2f}%")
```

OUTPUT

```
TRAINING PHASE (THRESHOLD FUNCTION)
Epoch 1
Training Accuracy : 56.26%
Bias: -0.1400
Weights (first 5 shown): [-0.01486398  0.24036508
 -0.45140647 -0.06606269 -0.13970254]
```

Epoch 2

Training Accuracy: 99.99%

Bias: 0.5555

Weights (first 5 elements): [0.0362536 0.1384141
-0.1415921489 -0.0521269 -0.13662041]

Epoch 3

Training Accuracy: 88.39%

Bias: 0.2555

Weights (first 5 elements): [0.0952883 0.0839972 -0.
-0.5329784 -0.0862065]

Epoch 4

Training Accuracy 88.51%

Bias: 0.2855

Weights (first 5 elements): [0.08549515 -0.0705311
-0.3638417137 -0.02776611 -0.0872208 2]

Epoch 5

Training Accuracy: 88.67%

Bias: 0.3155

Weights (first 5 elements): [0.0929 1211 -0.1364 4155
-0.2952514219 -0.02462014 -0.0852 6722]

Final Training Accuracy

Final Training Accuracy: 92.31%

Testing Phase

Final Test Accuracy: 93.36%